

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра радіоелектроніки та комп'ютерних систем

Звіт
Про виконання лабораторної роботи №1
З курсу “Машинне навчання”
«Лінійна регресія, оптимізація та статичний аналіз»

Виконав:
студент групи ФЕІ-44с
Мариняк А. П.

Перевірив:
доцент Сінькевич О.О.

Львів 2026

Мета роботи: Зрозуміти архітектуру лінійної регресії, навчитися самостійно реалізовувати алгоритми оптимізації, обирати інформативні ознаки, обчислювати метрики якості та перевіряти статистичні припущення моделі

Завдання:

Для досягнення поставленої мети необхідно виконати комплексний аналіз даних на прикладі набору Ames Housing Dataset, що включає первинне дослідження, відбір ознак на основі кореляції та правильну попередню обробку з дотриманням правил розбиття вибірки, кодування категорій та масштабування.

Після підготовки даних потрібно математично реалізувати функцію прогнозування, функцію втрат та обчислення градієнтів, використовуючи виключно векторизовані матричні операції бібліотеки numpy без використання стандартних циклів.

Наступним важливим кроком є розробка та порівняння трьох алгоритмів оптимізації: повного (Batch), стохастичного (SGD) та міні-пакетного (Mini-batch) градієнтного спуску, з обов'язковою візуалізацією процесу мінімізації функції втрат.

Після успішного навчання моделі вимагається провести оцінку її якості за допомогою коефіцієнта детермінації на тренувальній, валідаційній та тестовій вибірках. Заключна частина роботи передбачає перевірку ключових статистичних припущень лінійної регресії шляхом візуального аналізу залишків моделі, а також виконання додаткового експерименту для дослідження геометричного впливу масштабування ознак на стабільність роботи градієнтного спуску.

ТЕОРЕТИЧНІ ВІДОМОСТІ

Лінійна регресія та функція втрат

Лінійна регресія - це базова модель машинного навчання, яка описує залежність цільової змінної від однієї або кількох незалежних ознак за допомогою лінійної функції. У матричному вигляді модель записується як $y = Xw + b$, де X - матриця ознак, w - вектор ваг, а b - зміщення (bias). Для оцінки точності передбачень найчастіше використовується функція втрат Mean Squared Error (MSE), яка обчислює середнє значення квадратів різниць між реальними та прогнозованими значеннями

Головна задача процесу навчання полягає у знаходженні таких параметрів моделі, при яких значення MSE буде мінімальним

Оптимізація та градієнтний спуск

Мінімізація функції втрат здійснюється за допомогою алгоритму градієнтного спуску (Gradient Descent), який ітеративно оновлює ваги моделі в напрямку антиградієнта. Розмір кроку на кожній ітерації визначається параметром швидкості навчання (Learning Rate, alpha). Залежно від об'єму даних, що використовуються для одного кроку, виділяють три модифікації алгоритму:

- **Batch GD (повний):** обчислює градієнт на основі всієї навчальної вибірки, що гарантує стабільну збіжність, але вимагає значних обчислювальних та часових ресурсів
- **SGD (стохастичний):** використовує лише один випадковий приклад для кроку оптимізації, що значно пришвидшує роботу, але створює сильні коливання функції втрат
- **Mini-batch GD:** обробляє дані невеликими пакетами (наприклад, по 32 приклади), що забезпечує оптимальний компроміс між швидкістю виконання та стабільністю збіжності

Попередня обробка даних та оцінка якості

Для коректної та швидкої роботи градієнтного спуску критично важливим є масштабування ознак (стандартизація). Вона приводить усі дані до єдиного масштабу (із середнім 0 та дисперсією 1), роблячи геометричну поверхню функції втрат симетричною

Щоб уникнути витоку даних (Data Leakage) - ситуації, коли інформація з тестової вибірки впливає на процес навчання - масштабування параметрів застосовується виключно після розбиття датасету. Якість готової моделі оцінюється за допомогою коефіцієнта детермінації (R^2), який показує, яку частку дисперсії цільової змінної здатна пояснити побудована модель

Статистичні припущення моделі

Оскільки лінійна регресія є статистичним методом, її результати вважаються надійними лише за умови виконання базових припущень щодо залишків (помилки прогнозу)

До ключових припущень належать: наявність справжньої лінійної залежності в даних, нормальний розподіл залишків, гомоскедастичність (сталість дисперсії помилок на всьому діапазоні значень) та відсутність мультиколінеарності (сильної взаємозалежності між незалежними ознаками)

ХІД РОБОТИ

Етап 1. Дослідження даних (EDA), відбір ознак та попередня обробка

Для виконання роботи я обрав середовище Google Colab та завантажив файл train.csv (Ames Housing Dataset). Зчитавши дані за допомогою pandas, я провів аналіз кореляції Пірсона між числовими змінними та цільовою змінною SalePrice

Для побудови моделі я відібрав 3 числові ознаки з найвищою кореляцією (OverallQual - 0.79, GrLivArea - 0.70, YearBuilt - 0.52) та одну категоріальну - Neighborhood (район)

```
numeric_df = df.select_dtypes(include=[np.number])
correlations = numeric_df.corr()['SalePrice'].sort_values(ascending=False)

print("\nТоп-10 ознак, що найбільше корелюють з ціною (SalePrice):")
print(correlations.head(10))

#3 Формуємо спрощений датасет для нашої моделі
#Беремо 3 числові ознаки з високою кореляцією та 1 категоріальну
selected_columns = ['OverallQual', 'GrLivArea', 'YearBuilt', 'Neighborhood', 'SalePrice']
df_selected = df[selected_columns].copy()
```

Лістинг 1. Відбір ознак та формування спрощеного датасету

Далі здійснив розбиття даних на навчальну (Train - 60%), валідаційну (Validation - 20%) та тестову (Test - 20%) вибірки. Для категоріальної ознаки було застосовано OneHotEncoder, а для числових — StandardScaler.

Відповіді на теоретичні питання (Етап 1):

- **Чому розбиваємо дані до масштабування та що таке Data Leakage?**

Дані розбиваються до нормалізації, щоб уникнути витоку даних (Data Leakage). Якщо застосувати масштабування на всьому датасеті відразу, середнє значення та дисперсія розрахуються з урахуванням тестових даних.

Моделі "підгляне" статистику майбутніх даних, що штучно завищить її точність. Параметри скейлера (fit) мають обчислюватися виключно на Train

- **Як обробити нові категорії на валідації/тесті?**

Використав One-Hot Encoding з параметром `handle_unknown='ignore'`. Якщо на тесті з'явиться новий район, якого не було під час навчання, енкодер просто заповнить відповідні координати нулями, і програма продовжить роботу без помилок

Етап 2. Реалізація Лінійної Регресії засобами numpy

Створив функції для обчислення прогнозу, функції втрат (MSE) та обчислення градієнтів. Згідно з вимогами, не використовував цикли `for`, замінивши їх на векторизовані матричні операції `np.dot` та `np.sum`

```
def compute_gradients(X, y, w, b):  
    m = len(y)  
    predictions = predict(X, w, b)  
    error = predictions - y  
  
    #Векторизоване обчислення градієнтів без for  
    dw = (2 / m) * np.dot(X.T, error)  
    db = (2 / m) * np.sum(error)  
  
    return dw, db
```

Лістинг 2. Векторизоване обчислення градієнтів та MSE

При ініціалізації нульовими вагами початкова функція втрат (MSE) склала

Результат тестування:

Початкова функція втрат (MSE) при нульових вагах: 38048921903.62

Етап 3. Градієнтний спуск: три варіанти оптимізації

Розробив три окремі функції оптимізації: Batch GD (повний), SGD (стохастичний, 1 приклад) та Mini-batch GD (з розміром батчу 32)

Навчивши моделі впродовж 150 епох, побудував порівняльний графік

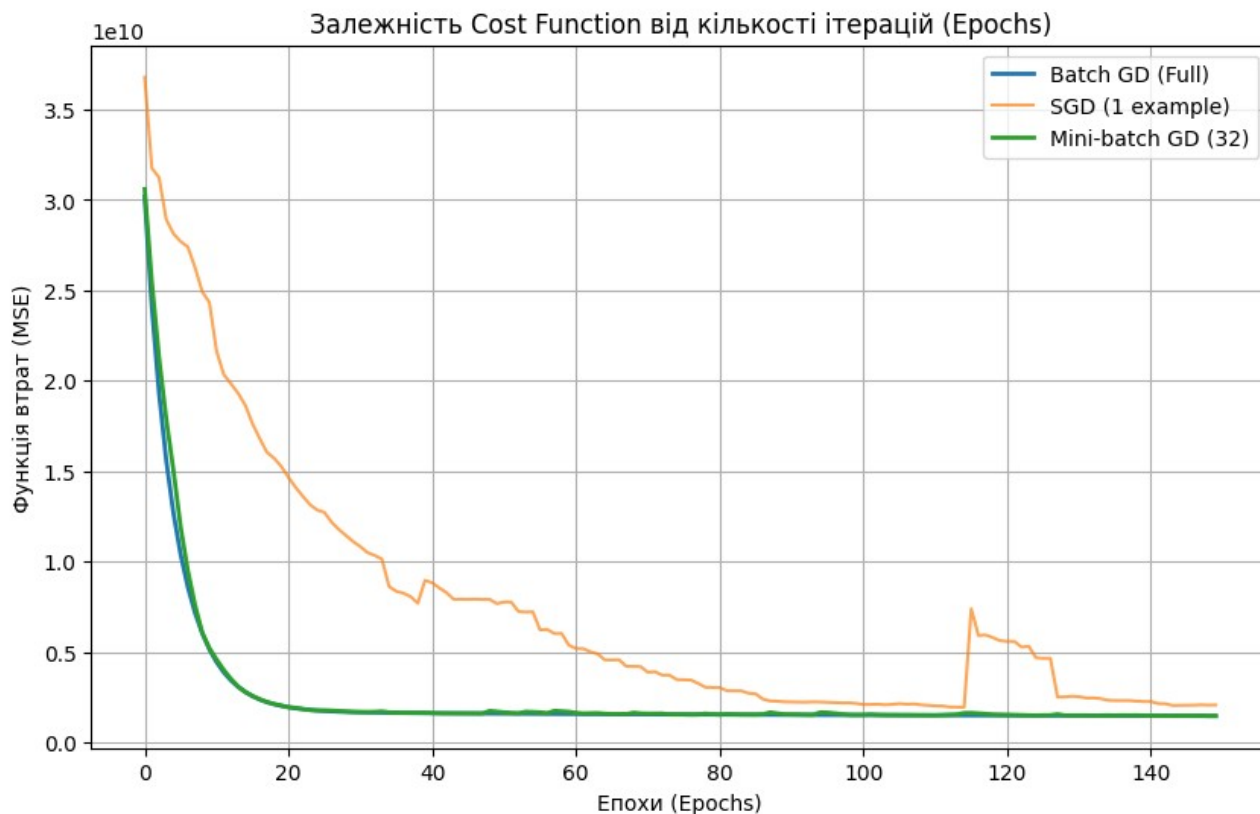


Рис. 1. Збіжність функції втрат для трьох алгоритмів оптимізації

Відповіді на теоретичні питання (Етап 3):

- **Який метод збігається найстабільніше?**

Найстабільнішим є Batch GD (синя лінія на Рис. 1), оскільки антиградієнт обчислюється точно по всій вибірці, і функція втрат плавно спадає.

- **Який метод має найбільші коливання?**

SGD (помаранчева лінія) має значні стрибки. Оскільки крок розраховується лише по одному випадковому прикладу, алгоритм сильно відхиляється від ідеального маршруту, хоча глобально спускається вниз

- **Вплив розміру batch:** Розмір батчу 32 (Mini-batch GD) є золотою серединою. Він згладжує коливання (зелена лінія майже повторює синю), але потребує значно менше ресурсів на кожну ітерацію, пришвидшуючи навчання
- **Вплив Learning Rate (alpha):** Правильний вибір alpha гарантує збіжність. Якщо крок занадто великий, алгоритм перестрибне мінімум і розійдеться (Cost прямуватиме до нескінченності). Якщо занадто малий - навчання триватиме занадто довго

Етап 4. Оцінка якості та коефіцієнт детермінації (R^2)

Реалізував функцію розрахунку R^2 засобами numpy та перевірів модель, отриману методом Mini-batch GD

```
def r_squared(y_true, y_pred):
    ss_res = np.sum(np.square(y_true - y_pred)) #Сума квадратів залишків
    ss_tot = np.sum(np.square(y_true - np.mean(y_true))) #Загальна сума квадратів
    return 1 - (ss_res / ss_tot)

#Для фінальної оцінки візьмемо ваги, отримані найкращим методом Mini-batch
best_w, best_b = w_mb, b_mb

#Прогнози для всіх вибірок
y_train_pred = predict(X_train_final, best_w, best_b)
y_val_pred = predict(X_val_final, best_w, best_b)
y_test_pred = predict(X_test_final, best_w, best_b)

#Обчислення  $R^2$ 
r2_train = r_squared(y_train, y_train_pred)
r2_val = r_squared(y_val, y_val_pred)
r2_test = r_squared(y_test, y_test_pred)
```

Лістинг 3. Обчислення коефіцієнта детермінації

Отримані метрики на Test:

R^2 градієнтний спуск : 0.8070

R^2 стохастичний: 0.7474

R^2 Mini-batch GD (Test): 0.7907

Відповіді на теоретичні питання (Етап 4):

- **Статистичний зміст R^2 :**

Метрика показує, яку частку дисперсії (розкиду) цільової змінної пояснює модель. В нашому випадку - 79%

- **Чи може R^2 бути від'ємним?**

Так, якщо сума квадратів помилок моделі перевищує загальну дисперсію даних.

- **Train vs Test (стан моделі):**

Показник R^2 на Test (0.79) не нижчий за показник на Train (0.72). Це свідчить, що модель чудово узагальнює нові дані і не має перенавчання (Overfitting)

Етап 5. Перевірка статистичних припущень

Обчислив залишки для тестової вибірки, побудував діагностичні графіки за допомогою matplotlib і seaborn, та провів тест Шапіро-Уїлка

```
import scipy.stats as stats
import seaborn as sns
import numpy as np
import matplotlib.pyplot as plt

#обчислення Прогнозів та залишків для тестової вибірки
predictions = predict(X_test_final, best_w, best_b)
residuals = y_test - predictions

plt.figure(figsize=(14, 10))
```

```
sample_size = min(500, len(residuals))
stat, p_value = stats.shapiro(np.random.choice(residuals, sample_size, replace=False))
print(f"Тест Шапіро-Уїлка (p-value): {p_value:.4f}")
```

Лістинг 4. Обчислення залишків та тест нормальності

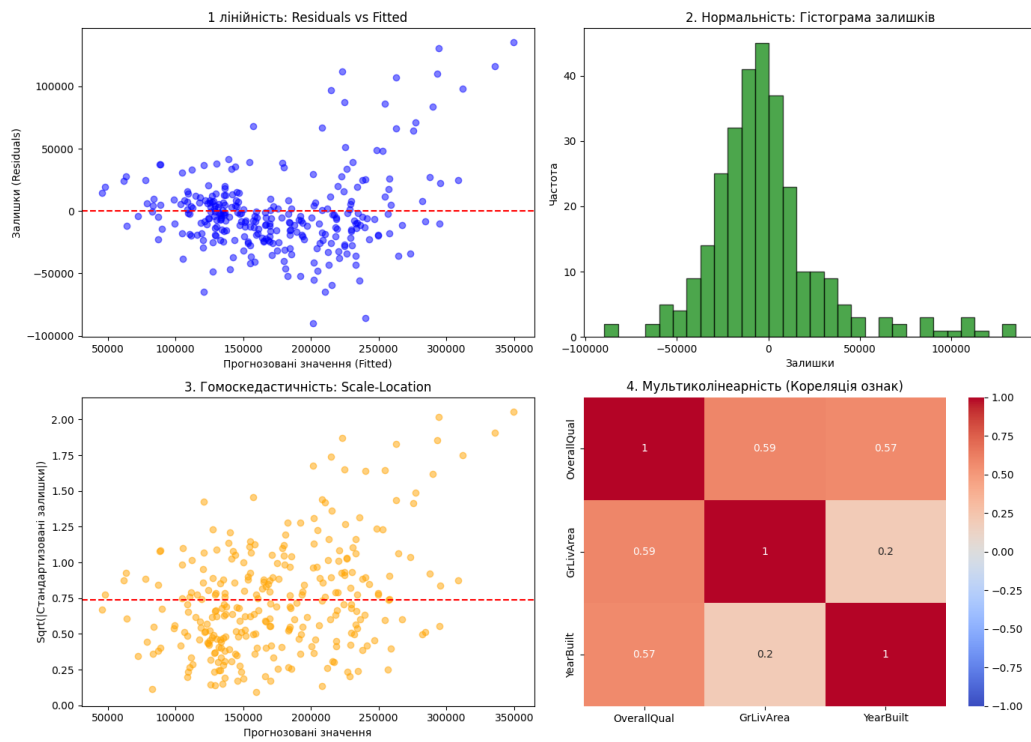


Рис. 2. Аналіз залишків та мультиколінеарності.

Висновки щодо припущень моделі (Етап 5):

- Лінійність:** Виконується (графік *Residuals vs Fitted* хаотичний, без патернів)
- Нормальність залишків:** Частково порушується, тест Шапіро-Уїлка повернув $p\text{-value} = 0.0000$. Гістограма має важкий правий хвіст через ексклюзивні дорогі будинки
- Гомоскедастичність:** Порушується, на графіку *Scale-Location* видно ефект "воронки" (гетероскедастичність)
- Мультиколінеарність:** Відсутня. Максимальна кореляція ознак склала 0.59, що безпечно

Етап 6. Завдання для глибокого розуміння (Експеримент А)

Провів експеримент із впливом масштабування, запустивши Mini-batch GD на стандартизованих та немасштабованих даних

```
X_train_unscaled = np.hstack((X_train[num_cols].values, train_cat))

#Для немасштабованих даних потрібен надзвичайно малий Learning Rate
lr_unscaled = 1e-7 #0.0000001
lr_scaled = 0.05 #Попередній стабільний крок

#Навчання моделі на немасштабованих даних
_, _, cost_unscaled = mini_batch_gd(X_train_unscaled, y_train, learning_rate=lr_unscaled, epochs=50, batch_size=32)

#Навчання моделі на масштабованих даних
_, _, cost_scaled = mini_batch_gd(X_train_final, y_train, learning_rate=lr_scaled, epochs=50, batch_size=32)
```

Лістинг 5. Експеримент впливу масштабування

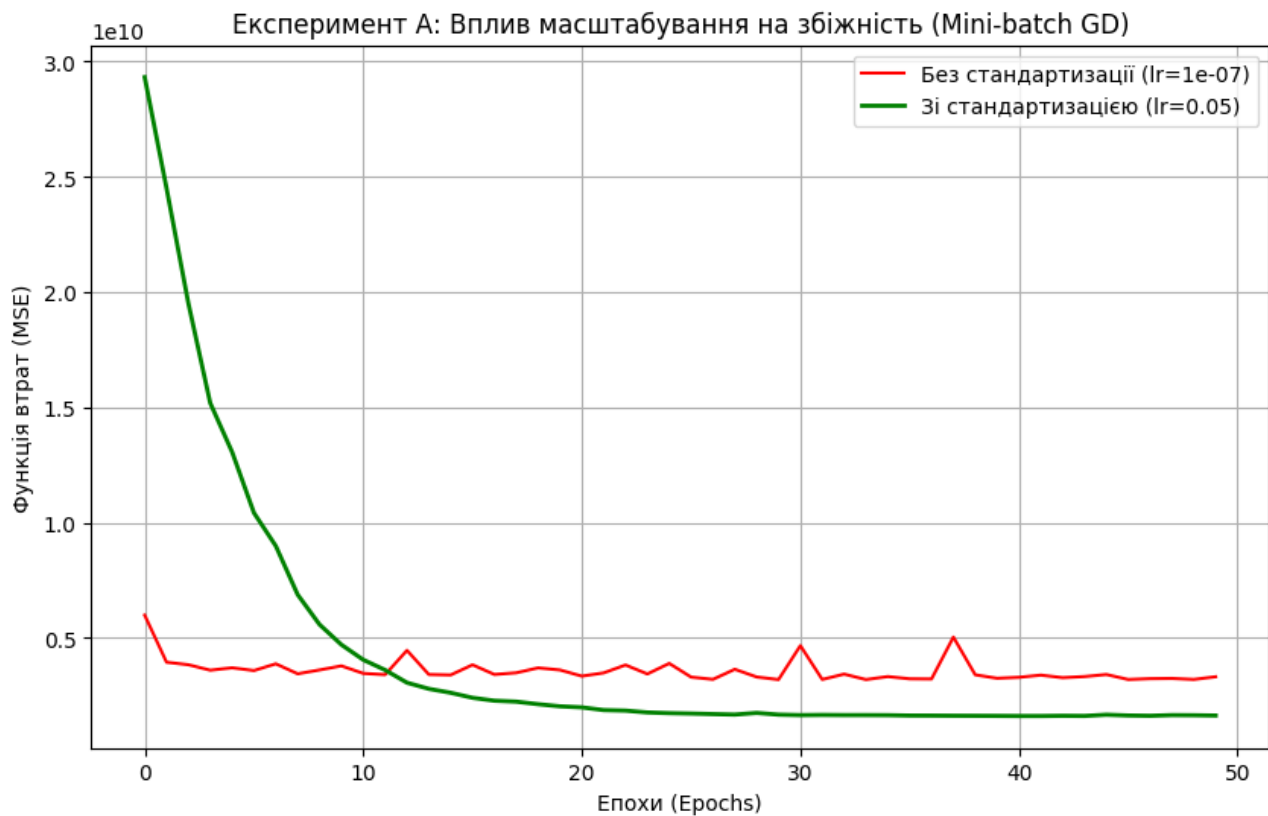


Рис. 3. Вплив масштабування на швидкість збіжності градієнтного спуску

Відповідь на теоретичне питання (Експеримент А):

Геометричне пояснення: Без стандартизації ознаки мають різні порядки величин. Геометрично поверхня функції втрат набуває форми витягнутого еліпса (вузького яру).

Антиградієнт відбивається від стінок, і алгоритм не може ефективно рухатися до центру. Довелося зменшити крок до 10^{-7} , що призвело до вкрай повільної збіжності (червона лінія)

Після стандартизації поверхня стала симетричною сферою, що дозволило взяти великий крок ($\alpha = 0.05$) і миттєво знайти оптимум (зелена лінія)

Висновок

Під час виконання лабораторної роботи розроблено модель лінійної регресії за допомогою векторизованих матричних операцій numpy.

Коректна попередня підготовка даних (масштабування та кодування строго після розбиття датасету) дозволила повністю уникнути витоку інформації

Порівняння алгоритмів оптимізації довело, що Mini-batch GD є найкращим компромісом між швидкістю та стабільністю. Модель показала високу здатність до узагальнення: коефіцієнт детермінації R^2 на тестовій вибірці склав 0.79 без жодних ознак перенавчання

Аналіз залишків підтвердив загальну адекватність моделі, хоча й виявив очікувану для ринку нерухомості гетероскедастичність (збільшення розкиду помилок для дорогих об'єктів)

Фінальний експеримент практично довів критичну роль стандартизації: приведення ознак до спільного масштабу геометрично вирівнює поверхню функції втрат, що дозволяє значно збільшити крок навчання та гарантує швидку збіжність градієнтного спуску